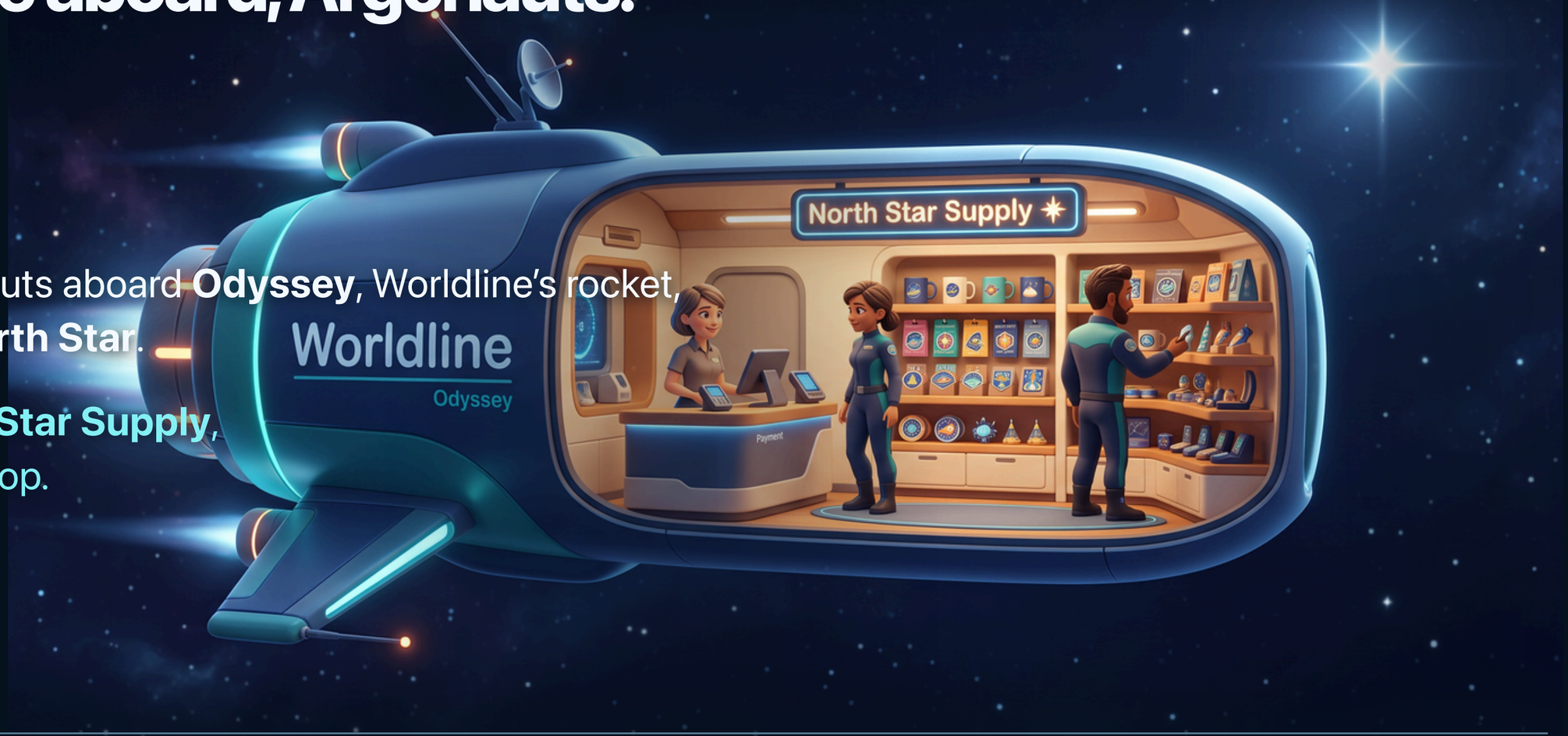
MISSION · THE BRIEFING

Welcome aboard, Argonauts.



You are Argonauts aboard **Odyssey**, Worldline's rocket, heading for **North Star**.

You run **North Star Supply**, the onboard shop.

The shop is ready. There is just one missing piece...

MISSION · THE MISSING PIECE

...We can reach North Star. We still can't sell a sticker.

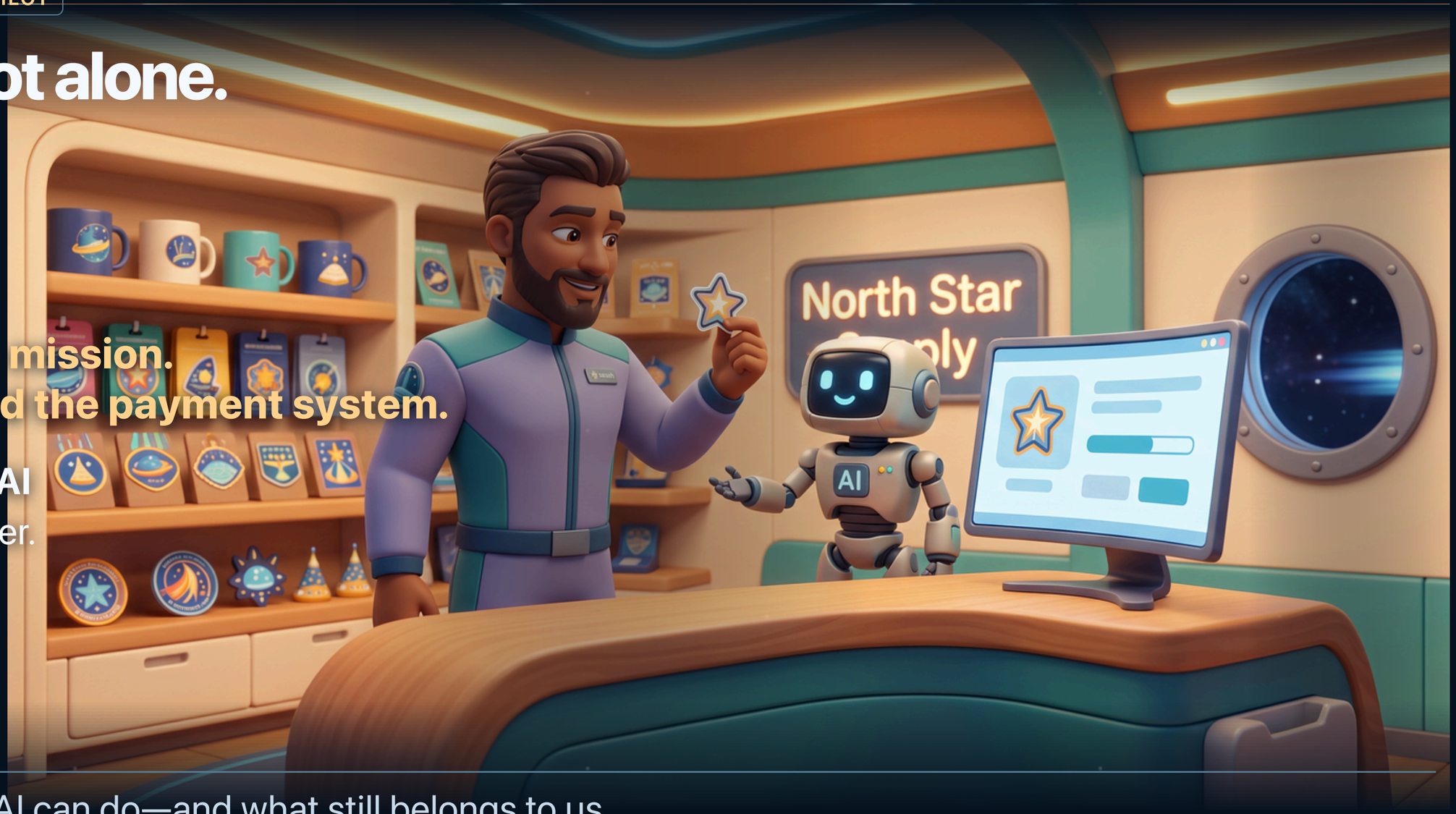
The shop has products and customers.
It still has no checkout.

We need a payment system to show the fees and check which payment methods are allowed.

That's your engineering mission.

MISSION · YOUR COPILOT

You're not alone.



You lead the mission.
AI helps build the payment system.

Time to recruit **AI**
as our co-builder.

Let's see what AI can do—and what still belongs to us.

MISSION · SET COURSE FOR NORTH STAR

Program of the afternoon

Consume agentic tools

Use agents to plan, code, review and test. Build new workflows around human approval and verifiable outputs.

Launch Sequence

14:05–14:17	Mission + Foundations + Inside LLM
14:17–14:25	From Chat to Harnesses
14:25–14:50	Pilot the Harness + live demo

Orbital Pause

14:50–15:05

Break

Build agentic architectures

Use AI to help engineer services with agents and orchestration, enriching their functional experience. Keep business rules in the service.

Build Your Crew

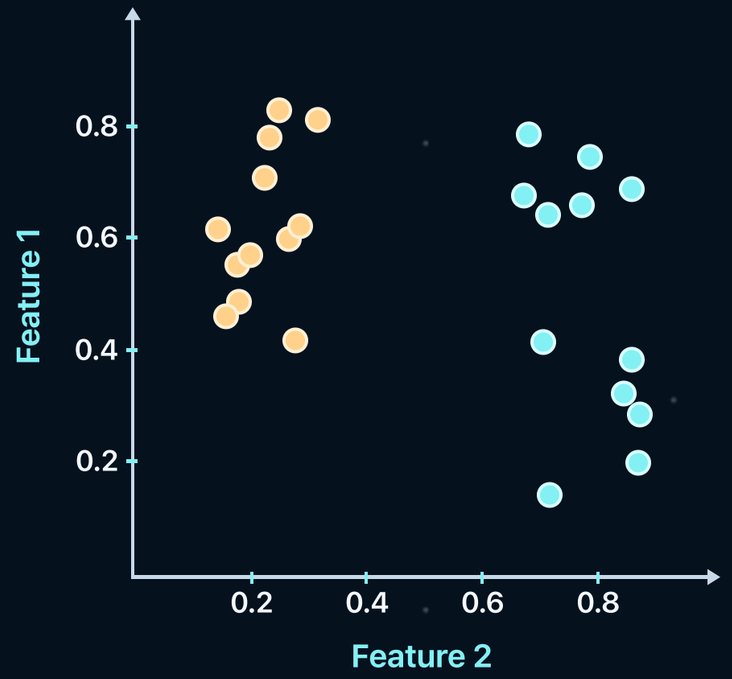
15:05–15:15	Connect the Crew
15:15–15:45	Recruit Your Agent
15:45–16:25	Build Your Crew
16:25–16:35	Mission Debrief

COURSE · FOUNDATIONS

How machines learn?

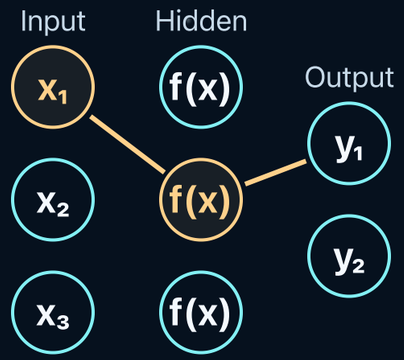
Machine Learning

Learn patterns from training examples.



Neural Networks

Weighted links + activation f(x).



Deep Learning

Neural networks with many layers.

◎ ★ Complex concepts

↔ ↗ + Combined features

| — / Simple features

COURSE · FOUNDATIONS

Natural Language Processing (NLP)

Tokenization maps text units to vocabulary IDs. Tokens are not necessarily words; the split depends on the tokenizer.

Input text

“The cat sits on the mat”

Language modelling turned out to do much more than we expected... #ChatGPT #2022 #OpenAI

COURSE · OPEN THE HOOD

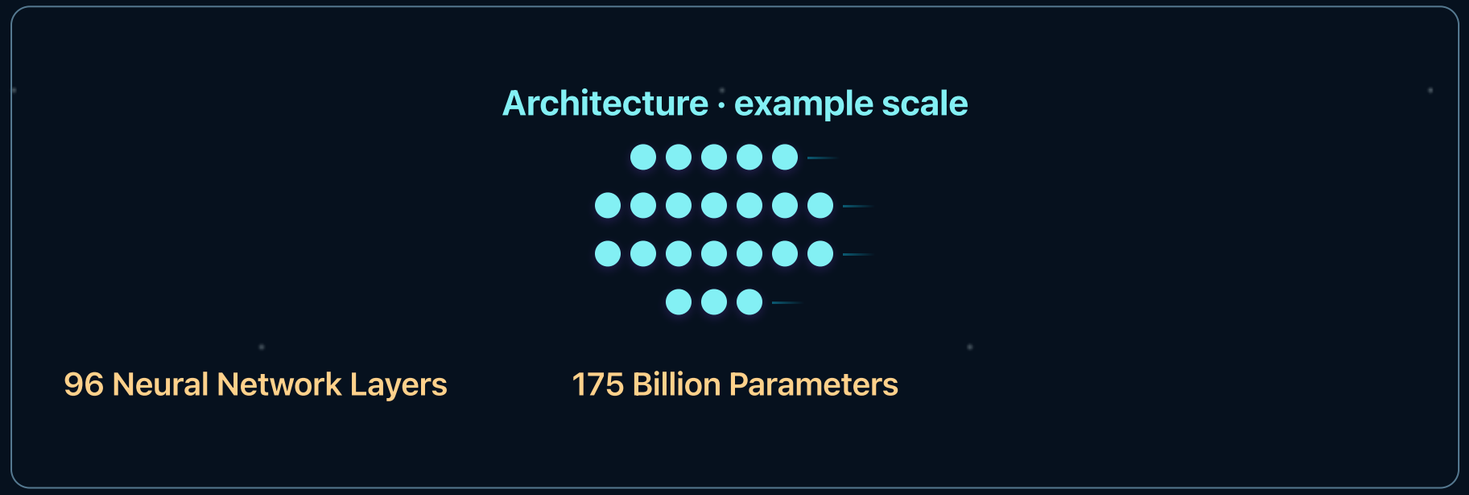
What is a Large Language Model?

- A large neural model trained to predict the next token.
- Training learns parameters.
- Chat uses them—it does not retrain the model.

COURSE · OPEN THE HOOD

Training at scale

Pre-training learns parameters from large text and code corpora by predicting the next token. GPUs make these updates scalable.



COURSE · OPEN THE HOOD

Tokenization and embeddings

Tokenization yields vocabulary IDs. The learned embedding matrix turns each ID into a vector.

Input text

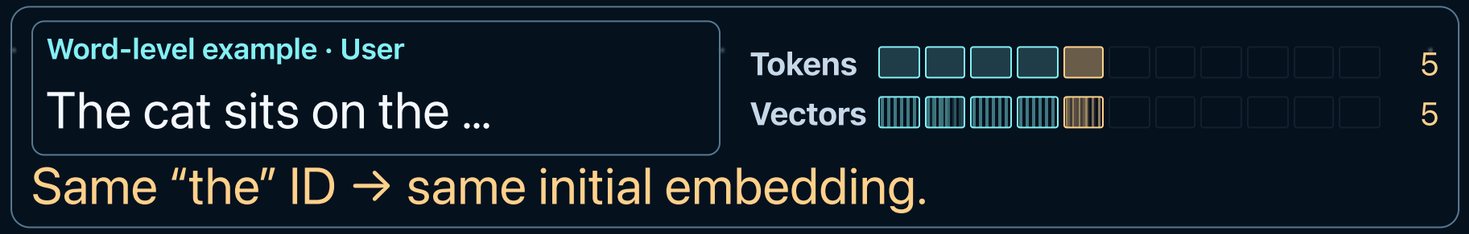
“The cat sits on the mat”

The same token ID selects the same initial vector. Model layers contextualize the full sequence.

COURSE · OPEN THE HOOD

Context grows with the conversation

Context is the information available to the model for its current response: instructions, selected conversation history and relevant data.

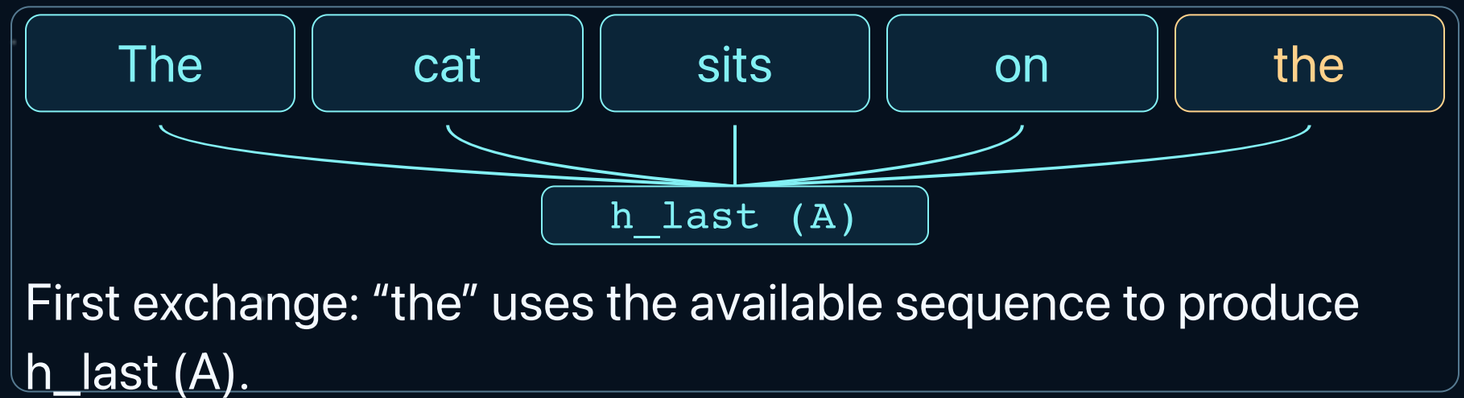


The context grows as the conversation continues: your messages, the model’s replies and tool results are added, within a finite context window.

COURSE · OPEN THE HOOD

Self-attention

Self-attention combines available token vectors and position information. Contextual representations change; learned weights stay fixed.



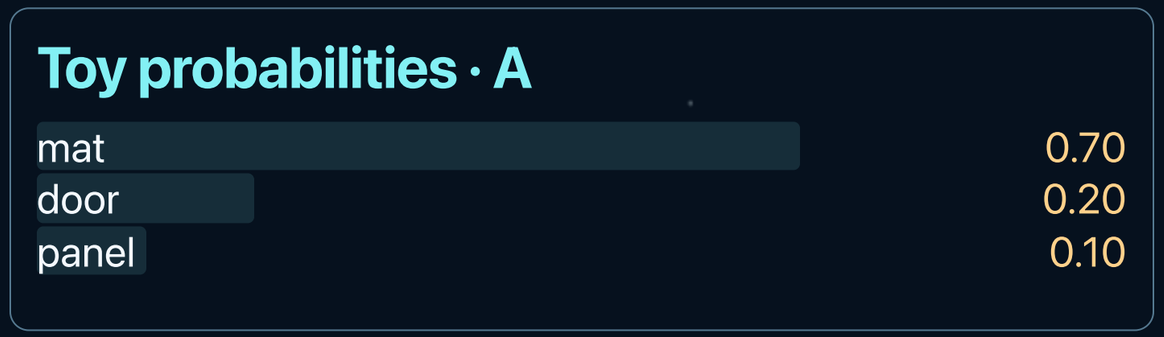
First exchange: “the” uses the available sequence to produce h_last (A).

COURSE · OPEN THE HOOD

Next-token prediction

The last contextual vector is projected to scores over the model's vocabulary, then converted into probabilities.

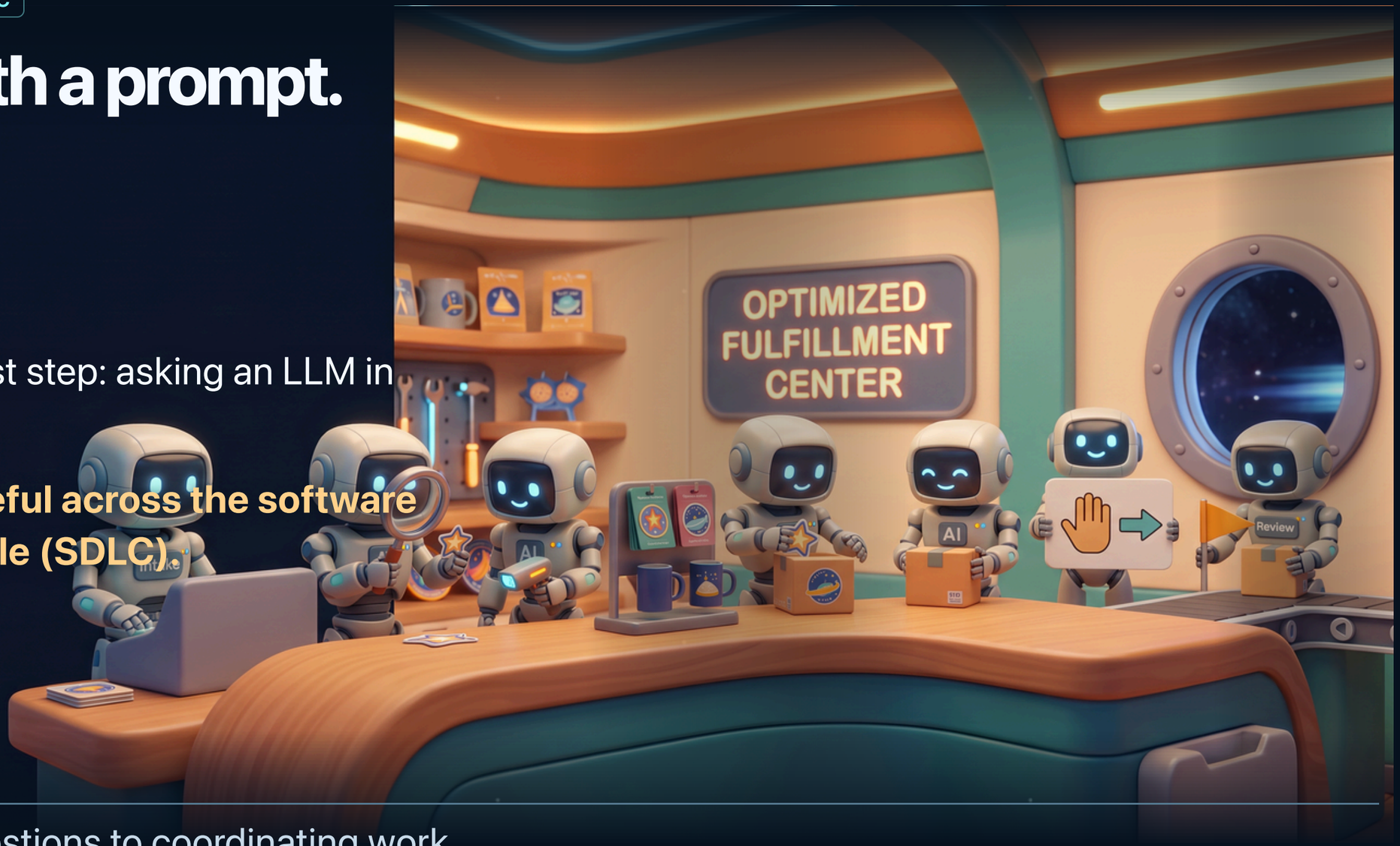
```
scores = out(h_last)
p = softmax(scores)
next_id = decode(p)
context.append(next_id)
```



Context: The cat sits on the ...
Append it; repeat until an end token or output limit.

COURSE · Generative AI in SDLC

It started with a prompt.



Prompting was our first step: asking an LLM in natural language.

It quickly proved useful across the software development lifecycle (SDLC).

From asking for suggestions to coordinating work.

COURSE · Generative AI in SDLC

Chat — manual execution

COURSE · Generative AI in SDLC

OpenAI · ChatGPT
30 Nov 2022 · Launch

Chat is the first way to use an LLM.

{} Generate
draft code, SQL or regular expressions.

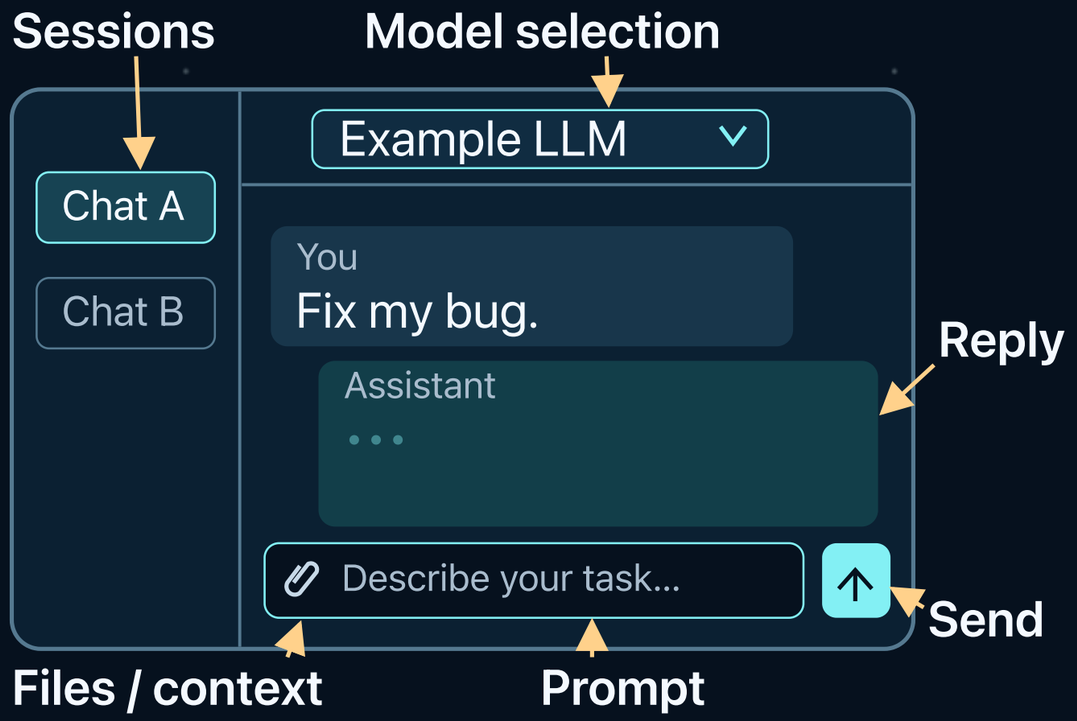
Explain / refactor
understand and improve existing code.

! Debug
analyse errors and propose corrections.

✓ Test
propose test cases and write unit tests.

Review
help inspect quality, performance and security.

≡ Document
draft comments and technical documentation.



COURSE · Generative AI in SDLC

IDE — assisted execution

COURSE · Generative AI in SDLC

IDE integration: closer to the project

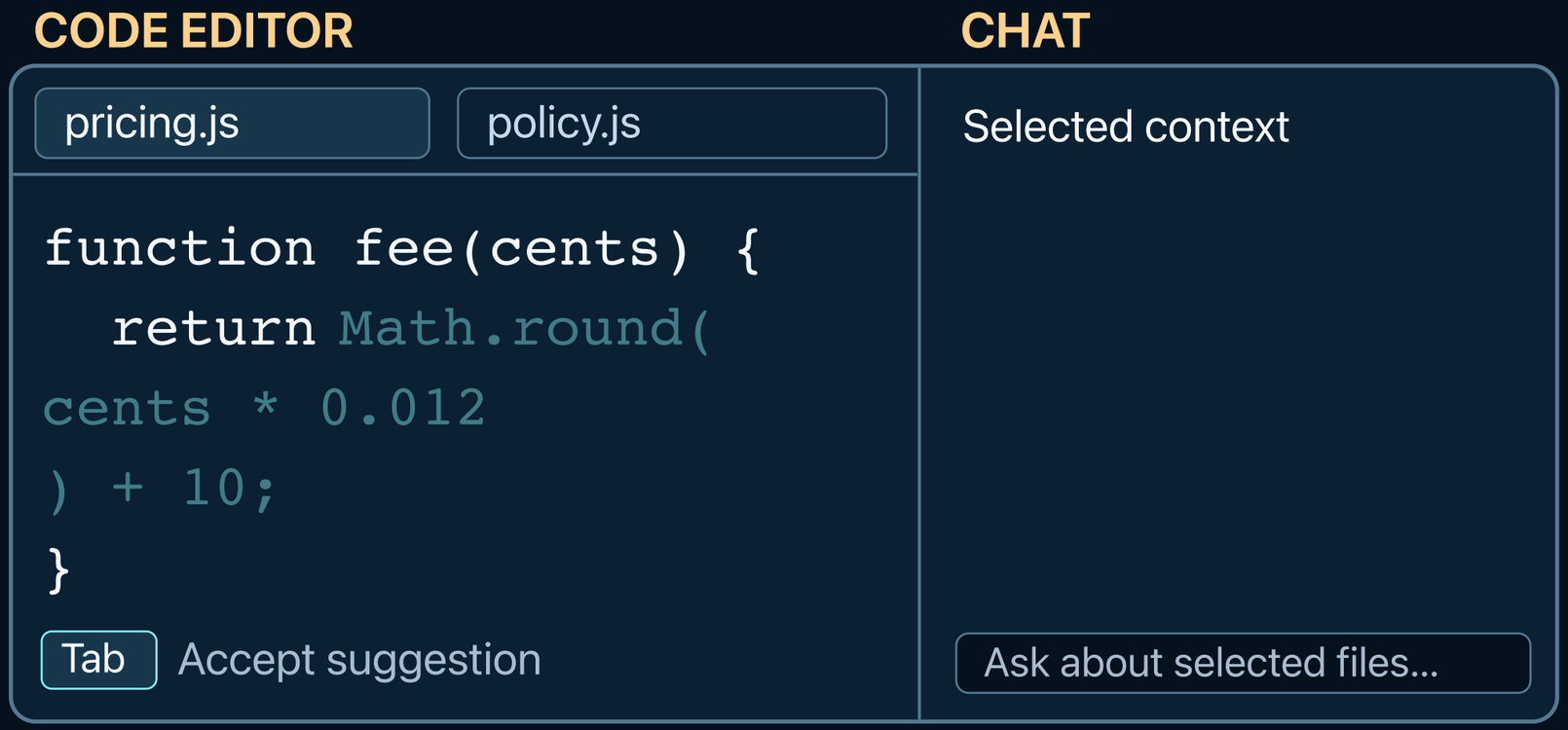
Copilot Chat · VS Code
20 Jul 2023 · Public beta

Completion
Suggest code where you work.

Context
Selected files—not the whole project.

Tool
Callable actions: read files or run tests.

Skill
Reusable instructions, resources and optional scripts.



Inline completion suggests code. Review before accepting.

COURSE · Generative AI in SDLC

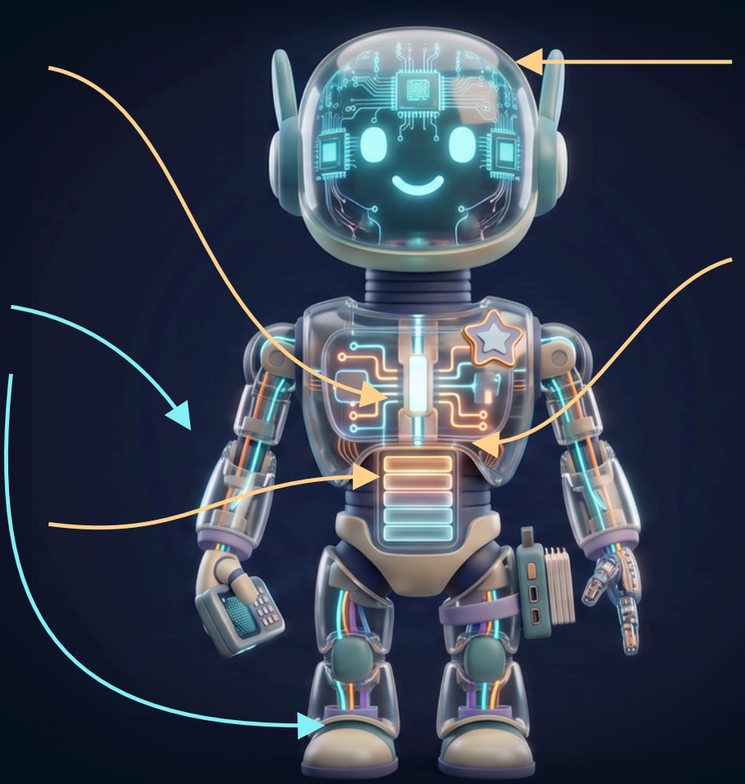
What is an AI agent?

A goal-driven model-and-tools loop. Persistent memory and RAG are optional.

Goal + instructions
Task + constraints

Tools
Read, edit, execute, call APIs

Runtime / loop
Act → observe → repeat



LLM
Choose the next action

Current context
Messages, state, results

Memory (optional)
Durable information

RAG (optional)
Retrieve external knowledge

COURSE · Generative AI in SDLC

Harness — supervised delegation

COURSE · Generative AI in SDLC

Harnesses turn decisions into actions

Anthropic · Claude Code
22 May 2025 · GA

Harness
Runtime for context, tools, permissions and limits.

Instructions file
Project rules: AGENTS.md or CLAUDE.md.

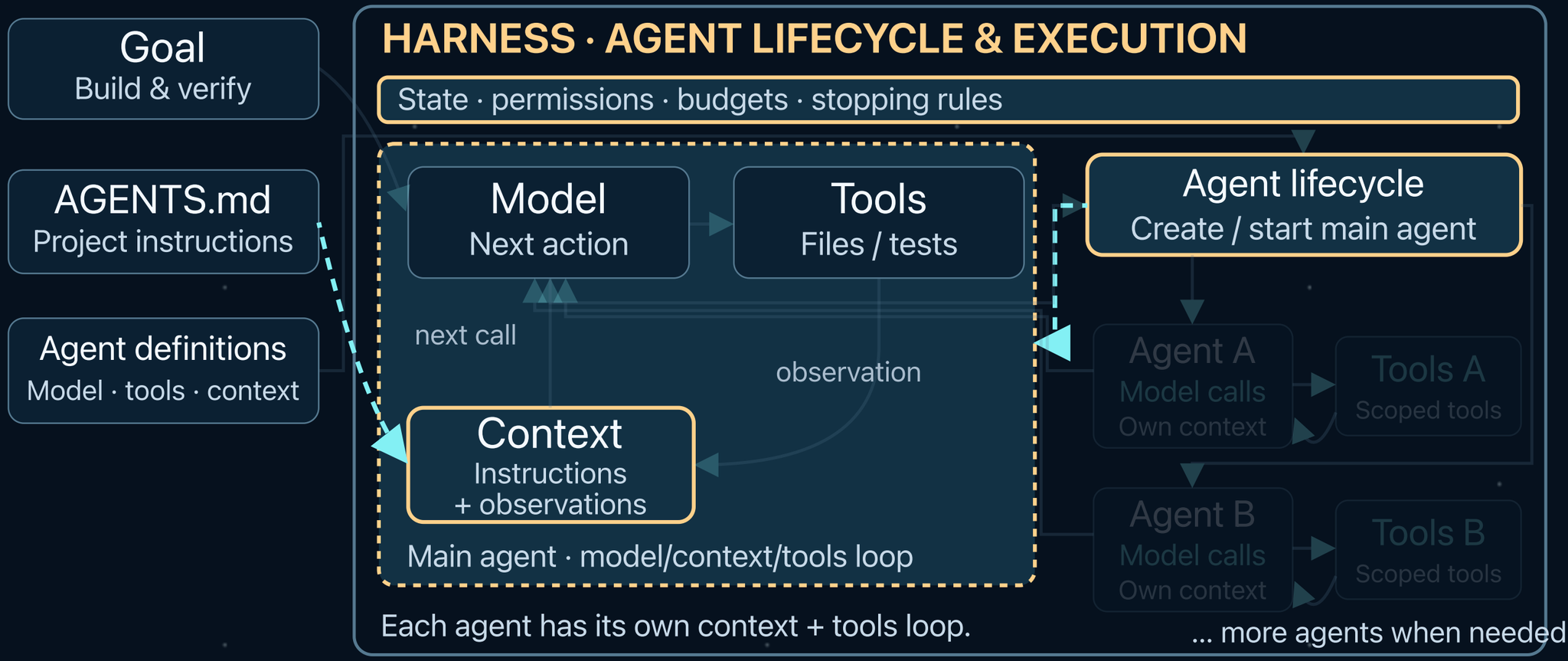
Orchestration
Optional coordination of independent agents.

```
✳ Claude Code                                   SIMULATED SESSION
north-star-supply/
❯ Add Wallet pricing and run the tests.

Model: Claude Sonnet                    Tokens (simulated): 0
Actions + evidence, not just suggestions. No commands executed.
```

COURSE · Generative AI in SDLC

Inside the harness: the agent loop



The harness configures the main agent: model, context and scoped tools.

COURSE · Generative AI in SDLC

Agentic is moving into every interface

SaaS

On-premises

Web

Desktop

Scaling

Agent loops multiply calls, latency, cost and concurrency. Set budgets, rate limits and stopping conditions.

Security

Tools multiply privileges and data exposure. Use scoped secrets, least privilege, approval gates and isolation.

MISSION · NEXT CHAPTER

Pilot the Harness

MISSION · CHECK IN

Your mission starts now.

Following along?

- Copilot licence active
- Copilot CLI installed + signed in

Watching? No Copilot setup required.

wrl.li/https://wrl.li/mission-onboard

MISSION · Functional

Payment catalogue

VISA · Wallet · Bank transfer

Rules and availability from the API.

Quote simulator

Enter an EUR amount and choose a method.
Show the fee, eligibility and rejection reasons.

Policy + API view

Display merchant rules and inspect raw HTTP requests, responses and call history.

Synthetic rules · for a €100 payment

Method	Fee rule / cap	Fee (€100)	Verdict
Visa	0.90% + €0.80 **Cap: 1.50% → €1.50**	€1.70	Cap exceeded
Mastercard	1.40% + €0.20 **Cap: €1.50 fixed**	€1.60	Cap exceeded
Wallet	1.20% + €0.10 **Cap: €1.50 fixed**	€1.30	**Eligible**
Bank transfer	0.40% + €0.10 **Cap: €1.50 fixed**	€0.50	Disabled

€1,000 example: Visa €9.80 < €15 cap → eligible.

MISSION · Technical

Frontend

Static HTML/CSS and vanilla JavaScript. Same-origin Fetch API; no React/Vue build pipeline.

Backend

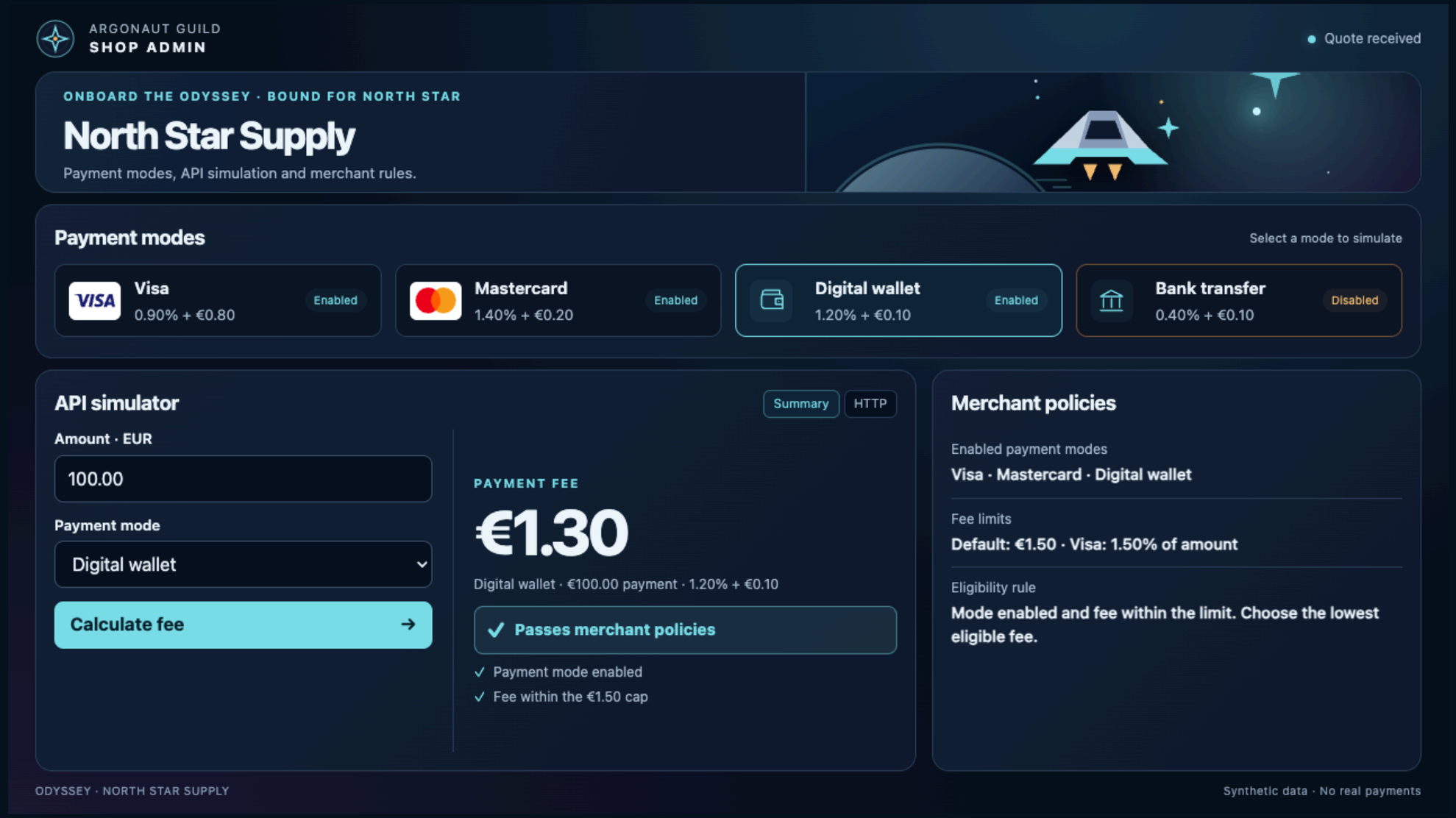
Node.js ≥22 + Express 5.3.0. pricing.js and policy.js own the calculations and validation.

Data + tests

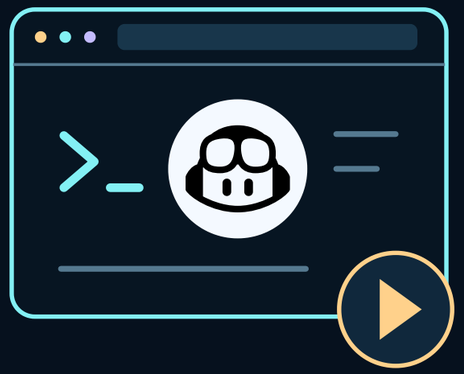
JSON fixtures, OpenAPI 3.1, Node's built-in test runner. No database, real payments or backend LLM calls.

Endpoint / input	Response · main JSON fields
GET /payment-methods	currency + methods[]
POST /quotes amount_cents payment_method	fee_cents + validation passed/failed, checks and failures
GET /merchant-policy Optional: amount_cents	allowed_payment_methods Fixed / percentage fee caps
GET /health	status + service + demo

MISSION · DESIGN MOCKUP



LIVE DEMO · TAKE THE CONTROLS



Copilot onboarding

`copilot`	Start in the project folder
`/login`	Sign in, if needed
`/model`	Choose the session model
`/help`	Show available commands
`/init`	Prepare repository guidance
`/plan`	Plan before coding

wrl.li/harness-demo-in

LIVE DEMO · SOLUTION

wrl.li/agent-reference

BREAK

Orbital Pause

Time to recharge.

We resume at 15:05.

COURSE · MODEL API

LLMs are also accessible through APIs

Request · illustrative

```
POST /v1/chat/completions
Authorization: Bearer <API_KEY>
{
  "model": "<model-id>",
  "messages": [
    {
      "role": "user",
      "content": "Hi"
    }
  ]
}
```

Response · excerpt

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "Hi, Argonauts!"
      }
    }
  ]
}
```

OpenAI-compatible Chat Completions · a de facto standard

`GET /v1/models` `POST /v1/embeddings`

MISSION · CONNECT THE CREW

The shop is built. Now recruit the crew.

North Star Supply has a payment service. Now give it agents that can use it.

The next mission

Turn a passenger's question into API calls and an evidence-based answer.

Our route

Connect tools → recruit Pricing → assemble the checkout crew.

COURSE · INSPECT THE CAPABILITY

Inspect one payment quote

The API owns the calculation and the shop rules.

Request

```
POST /quotes
{
  "amount_cents": 10000,
  "payment_method": "mastercard"
}
```

Expected response fields

```
HTTP 200
{
  "fee_cents": 160,
  "validation": {
    "status": "failed",
    "fee_within_limit": false
  }
}
```

> HTTP 200 means the quote was computed. Eligibility is a separate verdict.

COURSE · LAB STARTER KIT

Your agent-building kit

Colab Notebook

Run Python cells in your browser.

Personal Gmail account

LangChain Framework

Connect the model, tools and the agent loop.

Installed inside Colab

Mistral API

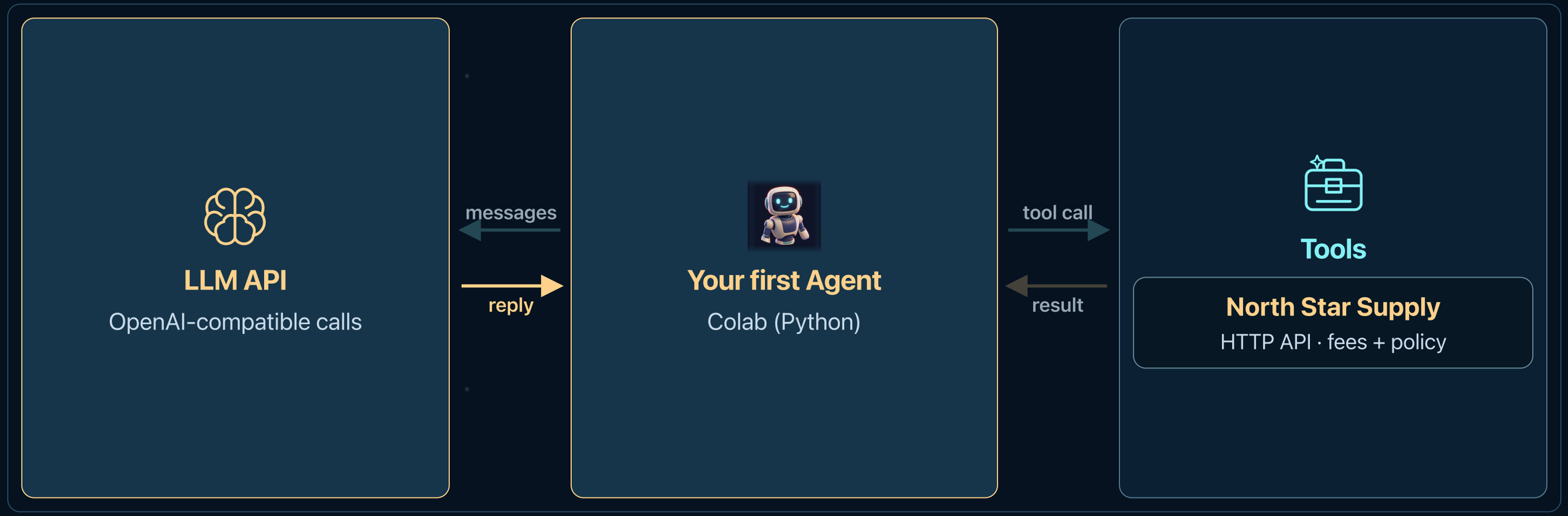
Provides model responses for the agent.

Account + API key

COURSE · TWO CONNECTIONS

Two APIs, two jobs

Your agent uses a model API and the shop API.



The model returns an answer—or a tool request.

COURSE · LANGCHAIN MODEL CLIENT

Connect a chat model with LangChain

Choose one provider. Reuse the client in your agent.

OpenAI

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(api_key=API_KEY, model="<openai-model-id>")
```

Mistral · workshop client

```
from langchain_mistralai import ChatMistralAI
llm = ChatMistralAI(api_key=API_KEY, model="ministral-8b-2512")
```

OpenRouter

```
from langchain_openrouter import ChatOpenRouter
llm = ChatOpenRouter(api_key=API_KEY, model="provider/model-id")
```

COURSE · EQUIP THE AGENT

North Star Supply services as tools

Tool request
Name + arguments

→

Python function
`@tool` + `api_request()`

→

North Star Supply API
`POST /quotes` → JSON result

```python
@tool
def get_payment_quote(amount_cents: int, payment_method: str):
    """Read a payment quote from North Star Supply."""
    body = {"amount_cents": amount_cents, "payment_method": payment_method}
    return api_request("POST", "/quotes", body)
```

MISSION · RECRUIT YOUR AGENT

Give the shop a Pricing Agent

Your passenger's request

"What is the Mastercard fee for a €100 purchase?"

Your agent's responsibility

Read the catalogue and request a quote. Return the fee—not a payment recommendation.

Success evidence

A real tool result contains **fee_cents: 160**. The answer explains **€1.60**.

Pricing

Model + instructions + two tools + feedback loop

LAB · YOUR PRICING MISSION

Build it. Run it. Inspect the trace.

HTTP tools
`list_payment_methods`
`get_payment_quote`

→

Pricing scope
Fees only · no validation

→

Run and inspect
€100 · Mastercard · tool trace

```
pricing_agent = create_agent(
    model=llm,
    tools=[list_payment_methods, get_payment_quote],
    system_prompt=PRICING_SYSTEM,
)
```

LAB · ONBOARDING

Prepare your first agent

North Star Supply services backend URL

https://…ngrok-free.app

TP1 · Colab
wrl.li/agent-tp1

TP1 · correction
wrl.li/agent-tp1-solution

Prompt
"What is the Mastercard fee for a €100 purchase?"

Expected output · €100

```
[Pricing] REQUEST get_payment_quote
  amount_cents=10000
  payment_method=mastercard
[Pricing] RESULT fee_cents=160
[Pricing] ANSWER €1.60
```

MISSION · BUILD YOUR CREW

The cheapest method is not always allowed

The passenger's new request

"Which payment method should the shop offer for €100?"

Pricing knows fees. Policy knows the shop rules. The DeepAgent needs both specialists.

Expected service facts at €100

Method	Fee	API verdict
Visa	€1.70	Cap exceeded
Mastercard	€1.60	Cap exceeded
Wallet	€1.30	**Eligible**
Bank transfer	€0.50	Disabled

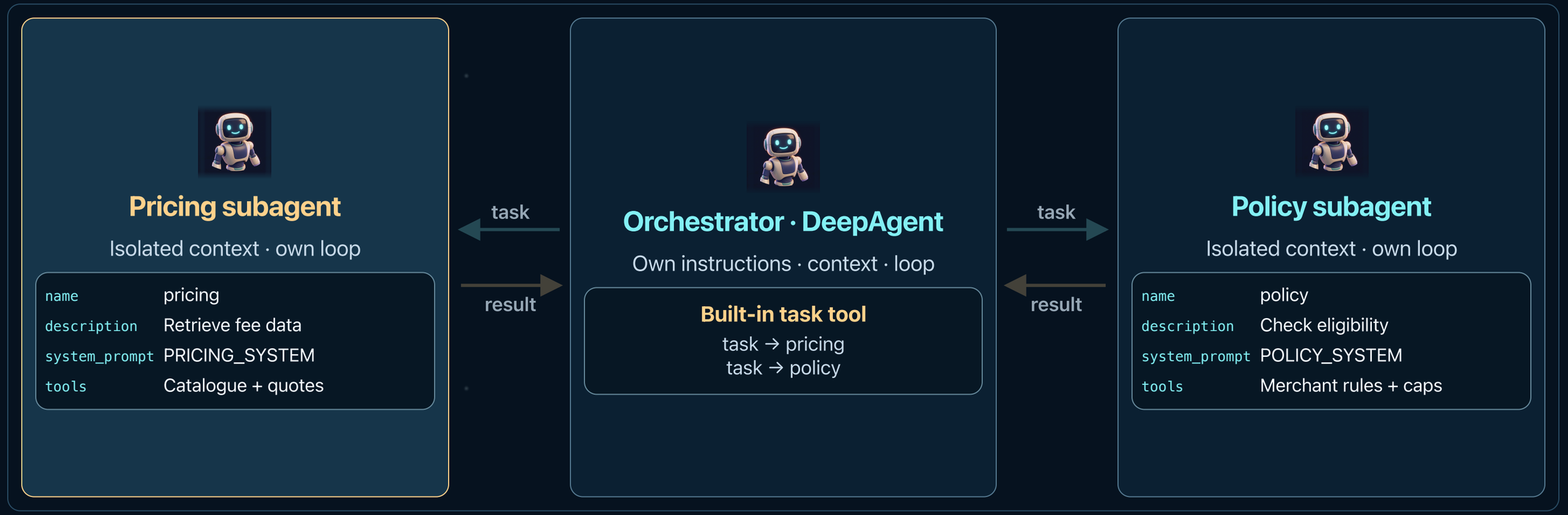
Success

Recommend **Wallet · €1.30**, with fee and policy evidence—not just the cheapest quote.

COURSE · ORCHESTRATION

A DeepAgent and two specialist subagents

DeepAgents coordinates two isolated specialist subagents.



Pricing uses its own instructions and tools to retrieve fee data.

LAB · YOUR CREW MISSION

Configure the subagents. Run the DeepAgent.

Subagent profiles
name · description · prompt · tools

→

DeepAgents harness
Delegation via the task tool

→

Workshop profile
General-purpose subagent disabled

```
checkout_agent = create_deep_agent(
    model=llm,
    system_prompt=ORCHESTRATOR_SYSTEM,
    subagents=[pricing_subagent, policy_subagent],
)
```

Open challenge · €1,000 Which payment method would you recommend? Support your answer with results from both Pricing and Policy agents.

LAB · ONBOARDING

Prepare your checkout crew

North Star Supply services backend URL

https://…ngrok-free.app

TP2 · Colab

wrl.li/agent-tp2

TP2 · correction

wrl.li/agent-tp2-solution

Prompt

"Which payment method should the shop offer for €100?"

Expected output · €100

```
[DeepAgent] REQUEST task → pricing
  [Pricing] RESULT API fee data
[DeepAgent] REQUEST task → policy
  [Policy] RESULT API merchant rules
[DeepAgent] ANSWER Wallet · €1.30
```

DEBRIEF · ARCHITECTURE CHOICE

One agent—or a crew?

One fully equipped agent

One loop · full pricing and policy tools.

Fewer calls · simpler trace

DeepAgent + two subagents

DeepAgent Pricing Policy

Scoped instructions, tools and contexts.

More calls · coordination overhead

Three agents teach responsibility boundaries—not a requirement for this tiny shop.

DEBRIEF · OPTIONAL CONNECTOR

MCP connects tools—not decisions

Agent runtime
MCP client

→

MCP server
`tools/list · tools/call`

→

North Star Supply API
Calculations + merchant rules

```python
from mcp.server import MCPServer
server = MCPServer("north-star-supply")
@server.tool()
def get_payment_quote(amount_cents: int, payment_method: str):
    """Expose a shop quote as an MCP tool."""
    body = {"amount_cents": amount_cents, "payment_method": payment_method}
    return api_request("POST", "/quotes", body)
```

DEBRIEF · MISSION COMPLETE

Explain the system—not just the answer

Consume agentic tools

Harness-assisted planning, coding and verification.

Build agentic architectures

Model client + HTTP tools + agent runtime.

Keep engineering control

Inspect traces. Bound tools, time, cost and data.

Workshop materials

wrl.li/agent-materials

DEBRIEF · BACK TO WORLDLINE

Thank you, Argonauts.

Bring agentic AI into your next Worldline task.

- Start small with an approved tool.
- Keep human control and verify the evidence.
- Share what you learn with the community.

More missions coming soon

`ai-learn.worldline.tech`

NOT LIVE YET